ODYSSEY: Artificial Intelligence Learning Path

Machine Learning & Deep Learning Theory Logbook

Rayan CHENNAOUI, IPSA Aerospace Engineering student, Contact: rayan.chennaoui@ipsa.fr
Focus: Physics-Informed Neural Networks & Mechanical Engineering

ABSTRACT

Objective. This living document traces the theoretical and practical acquisition of AI competencies, primarily following the Stanford/Andrew Ng curriculum.

Scope. Coverage includes Supervised Learning (Regression, Classification), Unsupervised Learning (Clustering, PCA), and advanced Deep Learning architectures applied to structural mechanics.

Methodology. Each module is validated through mathematical derivation, Python (NumPy) implementation, and "Physics-Informed" reasoning to ensure applicability to aerospace engineering.

1. Supervised Learning

The global goal of supervised learning, is to build a classifier that learns how to predict to outcomes $y \in \{y^{(1)}, \dots, y^{(m)}\}$ given a set of data points $x \in \{x^{(1)}, \dots, x^{(m)}\}$. For this chapter, the notations will be as follows : θ : parameters, m : number of trainig examples, x : input, y : output, (x, y) : training example, $(x^{(i)}, y^{(i)})$: i^{th} training example, n : number of features.

1.1. General concepts

1.1.1. Hypothesis function:

The hypothesis is denoted as h_θ and represents the chosen model. For a given input data $x^{(i)}$, the model's predicted output is $h_\theta(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_n x_n^{(i)} = \sum_{j=0} \theta_i * X_i$ ($h_\theta(x^{(i)})$ is the same as $h(x^{(i)})$, just different notation).

1.1.2. Least squared error:

In linear regression, we use the squared error (difference between outcome and hypothesis function : $(h_\theta(x^{(i)}) - y)^2$, and in order to minimize de least squared error, we want to minimize θ (parameters).

1.1.3. Cost function:

In order to do that, we create a cost function J, commonly used to assess the performance of a model, and is defined as follows:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

1.1.4. Gradient descent:

One of the most used algorithm is the gradient descent algorithm. It adjusts weights and biases to minimize the error between predicted and actual outputs. Techniques like backpropagation (that we will study afterwards) rely on Gradient Descent to propagate errors through layers and update parameters. By noting $\alpha \in \mathbf{R}$ the learning rate, the update rule for gradient descent is expressed with the learning rate and the cost function as follows:

$$\theta \xleftarrow{updated} \theta - \alpha \nabla J(\theta)$$

Can also be rewritten as:

$$\theta_j \xleftarrow{updated} \theta - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

We can derive for j for example:

$$\begin{aligned} \frac{\partial J}{\partial \theta_j} &= 2 \cdot \frac{1}{2} (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_\theta(x) - y) \\ &= (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\ &= (h_\theta(x) - y) x_j \end{aligned}$$

So for all the dataset:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

So we can apply the Gradient Descent algorithm as follows:

$$\theta_j \xleftarrow{updated} \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

So we have to repeat it until convergence. So at each step, update for $j = 0, 1, \dots n$ where n is the number of features.

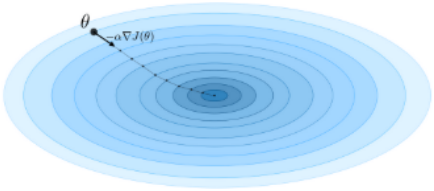


Fig. 1: Gradient Descent visualization (A. Ng, Supervised Learning)

Remark: Stochastic gradient descent (SGD) is updating the parameter based on each training example (one i for every j).

1.2. Notation of Operations & Properties

To handle optimization in high-dimensional spaces, we define the gradient with respect to a vector or a matrix.

1.2.1. The Gradient Vector:

For a parameter vector $\theta \in \mathbb{R}^{n+1}$, the gradient of the cost function $\nabla_\theta J(\theta)$ is defined as the vector of its partial derivatives:

$$\nabla_\theta J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}$$

1.2.2. Matrix Trace Properties:

The trace operator (tr) is essential for deriving gradients of quadratic forms. Based on the fundamental properties:

- **Cyclic Property:** $\text{tr}\, AB = \text{tr}\, BA$ and $\text{tr}\, ABC = \text{tr}\, CAB$.
- **Linear Gradient:** If $f(A) = \text{tr}\, AB$, then $\nabla_A \text{tr}\, AB = B^T$.
- **Quadratic Gradient:** $\nabla_A \text{tr}\, AA^T C = CA + C^T A$.

1.3. LWR

Parametric learning algo : fixed parameters θ_i ; Non-Parametric learning algo : parameters θ_i grows linearly with the amount of data set.

1.3.1. Locally Weighted Regression (LWR):

With linear regression, we were looking for a θ to minize the cost function $J(\theta)$. Locally Weighted Regression (LWR), weights each training example in its cost function by $w^{(i)}(x)$ which is defined with parameter $\tau \in \mathbf{R}$ (bandwidth parameter that controls the width of the neighborhood) :

$$w^{(i)}(x) = exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

So now we want to minimize:

$$\sum_{i=1}^{m} \mathbf{w^{(i)}(x)} \left(y^{(i)} - h_\theta(x^{(i)})\right)^2$$

In LWR, the weights $w^{(i)}$ determine the influence of each training example $(x^{(i)}, y^{(i)})$ when fitting the curve at a specific query point x. The value of $w^{(i)}$ ranges between 0 and 1. If the distance between the training example and the query point is small, the weight is large, meaning the model pays significant attention to that point. Conversely, if the distance is large, the weight becomes negligible.

- If $|x^{(i)} - x|$ is small, then $w^{(i)} \approx 1$.
- If $|x^{(i)} - x|$ is large, then $w^{(i)} \approx 0$.

As a result, only the terms multiplied by a weight near 1 (those close to our query point on the curve) stay relevant in the summation, while others are effectively neglected.

1.4. Probabilistic interpretation (likelihood...)

1.4.1. Logistic Regression:

We assume that the target variable $y^{(i)}$ and the inputs $x^{(i)}$ are related by the following equation:

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$$

where $\epsilon^{(i)}$ represents the error term (unmodeled effects or random noise). We assume that these errors are distributed according to a Gaussian (Normal) law:

$$\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$$

This implies that the probability density of the noise for each specimen is:

$$P(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right)$$

Since $\epsilon^{(i)} = y^{(i)} - \theta^T x^{(i)}$, the distribution of the target variable y given the parameters θ and inputs x is also Gaussian, with a mean of $\theta^T x$ and a variance of σ^2:

$$P(y^{(i)}|x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Note: The notation "$;\theta$" indicates that the distribution is parameterized by θ.

1.4.2. Likelihood Function:

The likelihood function $L(\theta)$ represents the probability of the entire dataset for varying values of the parameters θ, while the data itself remains fixed. It is defined as the product of the individual probabilities for all m training examples:

$$\begin{aligned} L(\theta) &= P(\mathbf{y}|X; \theta) \\ &= \prod_{i=1}^{m} P(y^{(i)}|x^{(i)}; \theta) \\ &= \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \end{aligned}$$

1.4.3. **Maximum Likelihood Estimation (MLE):**

To find the best parameters θ, we maximize the Likelihood function $L(\theta)$, which represents the probability of the entire dataset. By taking the log-likelihood and maximizing it, we find that the constant terms and the exponential vanish, leaving us with the requirement to minimize:

$$\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2$$

This proves that under the assumption of Gaussian noise, the Maximum Likelihood Estimation for θ is exactly the **Ordinary Least Squares (OLS)** solution (next paragraph).

1.4.4. **Log-Likelihood:**

To simplify the maximization process, we define the log-likelihood $\ell(\theta)$ by taking the natural logarithm of the likelihood function. This transforms the product of exponentials into a summation:

$$\begin{aligned}
\ell(\theta) &= \log L(\theta) \\
&= \log \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
&= \sum_{i=1}^{m} \left[\log \frac{1}{\sqrt{2\pi}\sigma} + \log \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)\right] \\
&= m \log \frac{1}{\sqrt{2\pi}\sigma} - \sum_{i=1}^{m} \frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}
\end{aligned}$$

Since the first term is a constant and the second term is negative, maximizing $\ell(\theta)$ is equivalent to minimizing:

$$\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2$$

This demonstrates that maximizing the likelihood under a Gaussian noise assumption leads directly to the OLS objective.

1.5. Classification

With classification, we want the hypothesis function to produce values from 0 to 1 ($h_\theta(x) \in [0, 1]$). In order to "force" this response, we can use the sigmoid function.

1.5.1. **Sigmoid function:**

The sigmoid function g, also known as logistic function, is defined as follows:

$$\forall z \in \mathbf{R}, g(z) = \frac{1}{1 + e^{-z}} \in]0, 1[$$

1.5.2. **Logistic regression & likelihood:**

We assume here that $y|x; \theta \sim \text{Bernoulli}(\phi)$. We have the following form:

$$\phi = p(y = 1|x; \theta) = \frac{1}{1 + \exp(-\theta^T x)} = g(\theta^T x)$$

Since $y \in \{0, 1\}$, we can combine the two cases ($P(y = 1)$ and $P(y = 0)$) into a single expression:

$$P(y|x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

So we can rewrite our likelihood expression thanks to the expression above:

$$\begin{aligned}
L(\theta) &= p(\mathbf{y}|X; \theta) \\
&= \prod_{i=1}^{m} p(y^{(i)}|x^{(i)}; \theta) \\
&= \prod_{i=1}^{m} h_\theta(x^{(i)})^{y^{(i)}} (1 - h_\theta(x^{(i)}))^{1-y^{(i)}}
\end{aligned}$$

Now we can look for $l(\theta) = \log(L(\theta))$ using properties of log:

$$l(\theta) = \log L(\theta) = \sum_{i=1}^{m} \left(y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right)$$

So now we want θ to maximize $l(\theta)$. In order to do that, we can use Batch Gradient Ascent.

$$\theta_j \xleftarrow{updated} \theta + \alpha \frac{\partial}{\partial \theta_j} l(\theta)$$

So we can developp:

$$\theta_j = \theta_j + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

where this sum represents $\frac{\partial}{\partial \theta_j} l(\theta)$.

1.5.3. **Newton's algo (faster for grad descent):**

Newton's algorithm is a numerical method that finds θ such that $l'(\theta) = 0$. Its update rule is as follows:

$$\theta \leftarrow \theta - \frac{l'(\theta)}{l''(\theta)}$$

we are looking to maximize theta by looking where the derivative $l'(\text{theta}) = 0$

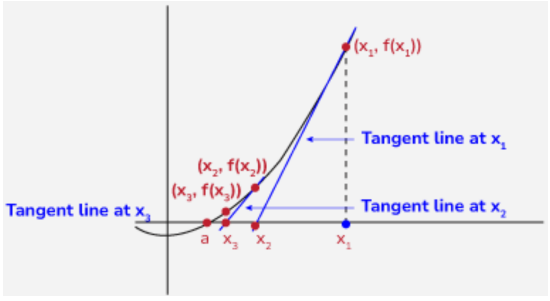


Fig. 2: Newton's algorithm method (GeeksForGeeks)

Remark:

– The multidimensional generalization, also known as the Newton-Raphson method, has the following update rule:

$$\theta \leftarrow \theta - (\nabla_\theta^2 \ell(\theta))^{-1} \nabla_\theta \ell(\theta)$$

– When θ is a vector ($\theta \in \mathbb{R}^{n+1}$), the update rule is:

$$\theta^{(t+1)} := \theta^{(t)} - H^{-1}\nabla_\theta \ell$$

Where H is the **Hessian matrix**, an $(n+1) \times (n+1)$ matrix:

$$H_{ij} = \frac{\partial^2 \ell}{\partial \theta_i \partial \theta_j}$$

The Newton's algorithm has one big advantage and one big disadvantage:

– Disadvantage: if theta is a vector, each iteration we have to derive every time, inverse big matrixes… gradient descent is better
– Advantages: if simple, the error doubles every iteration.

1.6. Perceptron

The following table summarizes the key differences in the hypothesis and activation functions between Logistic Regression and the Perceptron algorithm, as derived from the lecture notes.

Feature	**LR**	**Perceptron**
Activation Fct.	$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$
Hypothesis	$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$	$h_\theta(x) = g(\theta^T x)$
Output Type	Cont. prob. $\in [0,1]$	Discrete class $\in \{0,1\}$
Visual Form	Sigmoid	Hard step fct.

Table 1: Comparison of LR and Perceptron systems based on the activation thresholding.

The update function for the Perceptron algorithm remains structurally the same as in previous methods, although the hypothesis function $h_\theta(x^{(i)})$ represents a different mapping:

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

Where the term $(y - h)$ behaves as follows:

– It is 0 if the algorithm got the prediction right.
– It is +1 if the algorithm was wrong with $y = 1$.
– It is −1 if the algorithm was wrong with $y = 0$.

1.7. Exponential families

1.7.1. Definition & notation

To generalize the models we have seen (Linear and Logistic Regression), we introduce the Exponential Family of distributions. A class of distributions is in the exponential family if its probability density function (PDF) can be written in the following form:

$$p(y;\eta) = b(y)\exp\left(\eta^T T(y) - a(\eta)\right)$$

Where:

– y: The data (observed variable).
– η: The **natural parameter** (also called canonical parameter).
– $T(y)$: The **sufficient statistic** (often $T(y) = y$ in our cases).
– $b(y)$: The **base measure**.
– $a(\eta)$: The **log-partition function**.

The term $e^{-a(\eta)}$ acts as a normalization constant that ensures the distribution integrates to 1.

1.7.2. Usage Selection Guide

The choice of distribution depends on the nature of the target variable y:

– **Gaussian (Normal):** Used for real-valued outcomes ($y \in \mathbb{R}$), such as regression problems (e.g., predicting house prices or sensor values).
– **Bernoulli:** Used for binary classification ($y \in \{0,1\}$), where we predict the probability of a "yes/no" event.
– **Poisson:** Used for count data ($y \in \mathbb{N}$), such as the number of visitors to a website or the number of structural defects in a wing.
– **Geometric / Gamma / Exponential:** Used for positive real values ($y \in \mathbb{R}^+$), typically for modeling waiting times, duration, or volumes.

The following table summarizes the canonical components for common distributions:

Distrib.	η	$T(y)$	$a(\eta)$	$b(y)$
Bernoulli	$\log\left(\frac{\phi}{1-\phi}\right)$	y	$\log(1+\exp(\eta))$	1
Gaussian	μ	y	$\frac{\eta^2}{2}$	$\frac{1}{\sqrt{2\pi}}\exp\left(-\frac{y^2}{2}\right)$
Poisson	$\log(\lambda)$	y	e^η	$\frac{1}{y!}$
Geometric	$\log(1-\phi)$	y	$\log\left(\frac{e^\eta}{1-e^\eta}\right)$	1

Table 2: Common distributions to the exponential family form.

1.7.3. Four main PDF expressions

The following table summarizes the PDFs for the four main densities studied.

Distribution	**PDF**	**Type of Data**
Bernoulli	$p(y;\phi) = \phi^y(1-\phi)^{1-y}$	Binary $\{0,1\}$
Gaussian	$p(y;\mu) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{(y-\mu)^2}{2}\right)$	Real $\mathbb{R}$
Poisson	$p(y;\lambda) = \frac{e^{-\lambda}\lambda^y}{y!}$	Count $\mathbb{N}$
Geometric	$p(y;\phi) = (1-\phi)^{y-1}\phi$	Positive $\mathbb{R}^+$

Table 3: PDF of common distributions.

1.8. *Properties of the Exponential Family*

The exponential family (EF) distributions possess unique mathematical properties that guarantee the efficiency and stability of learning algorithms like Gradient Descent.

1.8.1. Optimization Properties: Concavity and Convexity

The form of the exponential family ensures that the optimization landscape is "well-behaved":

- **MLE with respect to η is Concave:** The Maximum Likelihood Estimation follows a concave curve (shaped like a mountain). This implies there is a single global maximum, ensuring that Gradient Ascent will always find the optimal parameters without getting stuck in local optima.

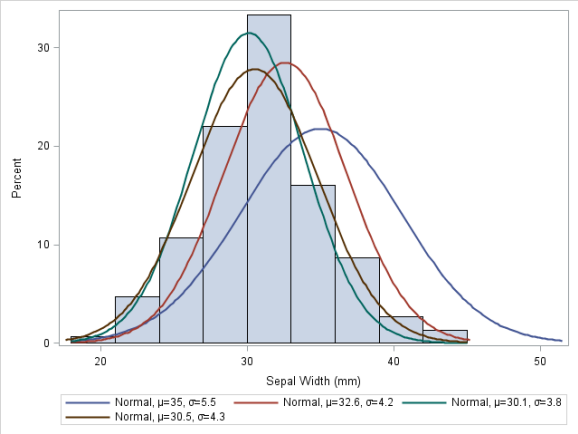


Fig. 3: Maximum Likelihood Estimation (Rick Wicklin)

- **NLL is Convex:** The Negative Log-Likelihood (often used as the Cost Function $J(\theta)$) is the opposite of the MLE. It is strictly convex (shaped like a bowl). Minimizing this function via Gradient Descent is guaranteed to converge to the unique global minimum.

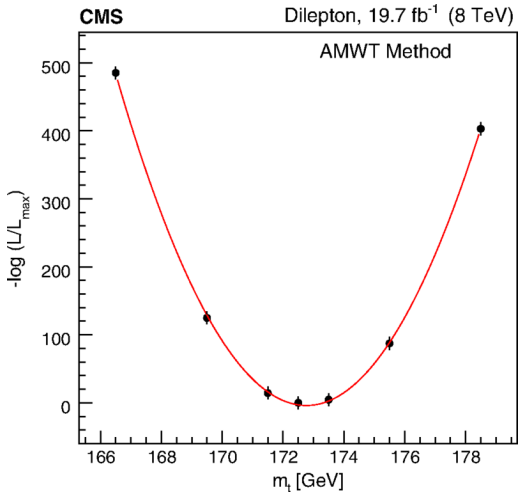


Fig. 4: Negative log-likelihood used for a dileption analysis (Vardan Khachatryan)

1.8.2. Mean & variance

A remarkable property of this family is that the mean and variance are encoded directly within the log-partition function $a(\eta)$. We can derive them by simple differentiation. The **Mean (Expected Value)** is the first derivative of $a(\eta)$ gives the mean of the distribution. The **Variance** is the second derivative of $a(\eta)$ gives the variance of the distribution.

$$E[y;\eta] = \frac{\partial}{\partial \eta} a(\eta); Var[y;\eta] = \frac{\partial^2}{\partial \eta^2} a(\eta)$$

Remark: This means that if you know the form of $a(\eta)$ for any distribution (Poisson, Gaussian, etc.), you automatically know its mean and variance without doing complex integrals.

1.9. *Gereralized Linear Model (GLM)*

Generalized Linear Models aim to predict a random variable y as a function of $x \in \mathbb{R}^{n+1}$. They unify various regression and classification models by relying on three fundamental assumptions.

1. **Distribution:** The conditional distribution of y given x belongs to the Exponential Family with natural parameter η:
$$y|x;\theta \sim \text{ExpFamily}(\eta)$$
2. **Prediction Target:** Given x, our goal is to predict the expected value of $T(y)$. In most cases, $T(y) = y$, so our hypothesis $h_\theta(x)$ is defined as:
$$h_\theta(x) = E[y|x;\theta]$$
3. **Linearity:** The natural parameter η and the inputs x are related linearly:
$$\eta = \theta^T x$$

Remark: Ordinary Least Squares (Gaussian) and Logistic Regression (Bernoulli) are special cases of GLMs.

The following diagram illustrates how the linear model connects to the exponential family to produce a prediction:

$$\text{Input } x \xrightarrow{\text{Linear Model: } \theta^T x} \eta \xrightarrow{\text{Exp. Family Params}} E[y;\eta] = E[y;\theta^T x]$$

Where θ is the parameter vector learned through optimization algorithms like **Gradient Descent**.

So to resume, we have 3 types of parameters: Model parameter θ, Natural parameter η and the Canonical parameters $(\phi, (\mu, \sigma), \lambda)$. We can easily find η with $\theta^T x$, and we can find the canonical parameters thanks to the link function g, where :

- Gaussian: $g(\eta) = \frac{1}{1+e^{-\eta}}$
- Bernoulli: $g(\eta) = \eta$
- Poisson: $g(\eta) = e^{\eta}$

Remark:

$$h_\theta(x) = E[y|x;\theta] = \phi = \frac{1}{1+e^{-\eta}} = \frac{1}{1+e^{-\theta^T x}}$$

This shows that the sigmoid function is the canonical response function g for the Bernoulli distribution.

1.9.1. **Softmax regression**

A softmax regression, also called a multiclass logistic regression, is used to generalize logistic regression when there are more than 2 outcome classes. Here is an example with 3 classes:

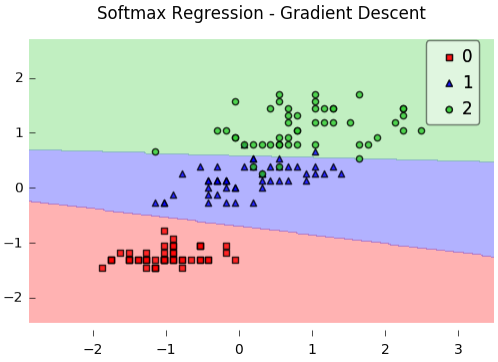


Fig. 5: Softmax regression example for 3 classes (mlxtend)

So for each point of the dataset, we'll know if its θ positive (and how positive?), negative (and how negative?), so we know in which class it is more probable that it is in.

After that, given these parameters $\theta^T x$ we can build a bars graph with all the classes to visualize the probability, then we exponential with $exp(\theta^T x)$, and finally we normalize this so the sum of all probabilities adds up to 1, this is the **Softmax Regression**:

$$\phi_i = \frac{exp(\theta_i^T x)}{\sum_{j=1}^{K} exp(\theta_j^T x)}$$

Where j represents the classes.

Finally, we would like to visualize a bar graph with 1 if the condition is true (data set in class j), 0 else. In order to do that, we minimize the cross-entropy between the truth distribution $\hat{P}$ and the predicted distribution P (Softmax output).

For a single example, where $\hat{y}$ is the true label (represented as a **one-hot vector**), the loss is defined as:

$$J(\theta) = -\sum_{i=1}^{K} \hat{y}_i \log(p_i) = -\log(p_{\text{correct class}})$$

The total cost function to minimize is the average cross-entropy over the entire training set:

$$J(\theta) = -\frac{1}{m} \sum_{j=1}^{m} \sum_{i=1}^{K} \mathbb{1}\{y^{(j)} = i\} \log \left(\frac{\exp(\theta_i^T x^{(j)})}{\sum_{l=1}^{K} \exp(\theta_l^T x^{(j)})} \right)$$

Where $\mathbb{1}\{\cdot\}$ is the indicator function, which equals 1 if the condition is true and 0 otherwise.

1.10. Gaussian Discriminant Analysis

1.10.1. **General concept**

The generative model approach of the Gaussian Discriminant Analysis (GDA) is the "opposite" of discriminative model (Linear Regression). Indeed, with GDA we are looking to estimate $p(x|y)$ to then deduce $p(y|x)$ thanks to Naive Bayes.

To understand this, here is an explanation with the email spams example:

- First we ask ourselves "if a spam is created, what are its characteristics (words, sentences...) ?", this is $p(x|y)$.
- By searching for $p(x|y)$, we learn about the Gaussian distribution of this particular class x (we don't have these informations in linear regression).
- Then, knowing $p(x|y)$ and $p(y)$ we can find $p(y|x)$ using Naive Bayes:

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

So this approach is very interesting because given a little dataset, we can elaborate a model that attributes a class for each input, whereas in linear regression, we must have a large dataset in order to create a separation between the two classes using gradient descent for example.

1.10.2. **Definition of GDA**

Now, it's important to understant that we are going to talk about multivariables Gaussian.

For GDA, we first drop the $x_0 = 1$ convention. We suppose $x \in \mathbf{R^n}$, and we assume that $p(x|y)$ is Gaussian (it will later allow us to use the means and covariance of the classes to "know how to regroup them").

We assume the following distributions for the multivariables Gaussian model:

- $y \sim \text{Bernoulli}(\phi)$
- $x|y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$
- $x|y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$

Where $z \sim \mathcal{N}(\mu, \Sigma)$ implies:

$$E[z] = E_z = \mu \quad \& \quad \text{Cov}(z) = E[(z-\mu)(z-\mu)^T] = E[zz^T] - (Ez)(Ez)^T$$

We can know define the PDF for a Gaussian, that's using the Mahalanobis distance in order to calculate the distance from one point of a class to the mean point of this class :

$$p(z) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right)$$

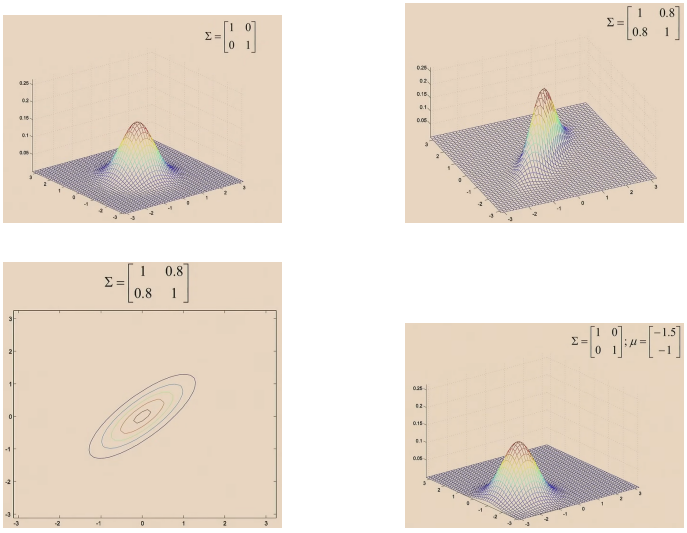


Fig. 6: Visual comparison of Multivariate Gaussian distributions with varying covariance matrices Σ, and means μ.

1.10.3. **Joint Likelihood**

So here are the two PDF for each case ($k = 0$ means $y = 0$ not a spam and $k = 1$ means $y = 1$ it is a spam), with the same covariance:

$$p(x|y_k) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k)\right)$$

Where $(\mu_1, \mu_2 \in \mathbf{R^n}, \Sigma \in \mathbf{R^{nxn}}, \phi \in \mathbf{R}$.

Now we can also build $p(y)$, which is ϕ if $y = 1$, and $1 - \phi$ else:

$$p(y) = \phi^y(1-\phi)^{1-y}$$

These two elements above allow us the find the **Joint likelihood**, given the training set $\{(x^{(i)}, y^{(i)})\}_{i=1}^m$:

$$\begin{aligned}\mathcal{L}(\phi, \mu_0, \mu_1, \Sigma) &= \prod_{i=1}^{m} p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\ &= \prod_{i=1}^{m} p(x^{(i)}|y^{(i)})p(y^{(i)})\end{aligned}$$

Here we are trying to find the parameters $(\phi, \mu_0, \mu_1, \Sigma)$ to minimize $(p(x^{(i)}, y^{(i)})$.

Before going further, it is essential to distinguish between the joint probability $p(x^{(i)}, y^{(i)})$ and the conditional probability$p(y^{(i)}|x^{(i)})$. In our generative model, $p(x^{(i)}, y^{(i)})$ represents the probability of observing a specific pair: that a given message has the content $x^{(i)}$ (words in spam list...) AND belongs to the class $y^{(i)}$ (being a spam). Using the chain rule of probability, we calculate it as:

$$p(x^{(i)}, y^{(i)}) = p(x^{(i)}|y^{(i)})p(y^{(i)})$$

This reflects the generative process: we first determine the class ($p(y)$) and then "generate" the features associated with that class ($p(x|y)$).

Whereas in discrimination, we were trying to find a parameter θ to minimize $p(x^{(i)}|y^{(i)})$:

$$\mathcal{L}(\theta) = \prod_{i=1}^{m} p(x^{(i)}|y^{(i)}; \theta)$$

1.10.4. **Maximum Likelihood Estimation (MLE)**

Using MLE, we can find the right parameters to minimize $\mathcal{L}(\phi, \mu_0, \mu_1, \Sigma)$:

First let's define the activation function:

$$\mathbb{1}_A(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases}$$

Then:

$$\phi = \frac{\sum_{i=1}^m y^{(i)}}{m} = \frac{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 1\}}{m}$$

$$\mu_0 = \frac{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 0\}x^{(i)}}{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 0\}}$$

If it's not a spam, then +1 with the activation function in the denominator, and $+1 * x^i$ in numerator, so we end up isolating only the cases where the mail is not a spam, where x^i is the feature vectors for specimens with $y = 0$. The same goes for μ_1.

$$\Sigma = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Here, the "shared" covariance matrix puts ellipse around those mean points, we need to multiply by itself transposed because they are vectors, we want a value to draw the projection for each points, link them, and draw a circle, or ellipse or other geometric forms around the mean points of the MLE to create a "zone" for each class.

- If Σ is the identity matrix I, the zone is a circle,
- if Σ is diagonal, the zone is an axis-aligned ellipse,
- if Σ contains non-zero off-diagonal terms, the ellipse is rotated, showing correlation between features.

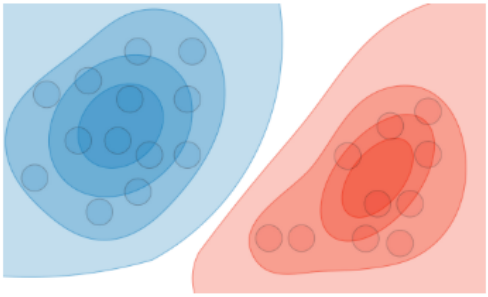


Fig. 7: Representation of two classes, using GDA

$$\begin{aligned}\arg\max_y p(y|x) &= \arg\max_y \frac{p(x|y)p(y)}{p(x)} \\ &= \arg\max_y p(x|y)p(y)\end{aligned}$$

Now we want to visualize using a problem. Let's assume we have a data set of 6 black squares and 6 red circles along the x axis.



Fig. 8: Data set of 6 black squares and 6 red circles

We can represent the distribution of theses 2 classes using two Gaussian curves.

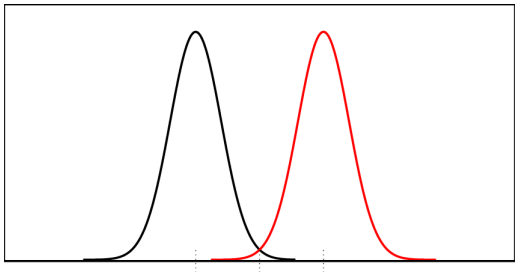


Fig. 9: Two Gaussian curves to visualise sigmoid function

Assuming that $(p(y = 1|x))$ represents the case where it is a circle, we can now draw a curve to represents the probability for

each point allong the x axis of it being a red circle, this is exactly a sigmoid function.

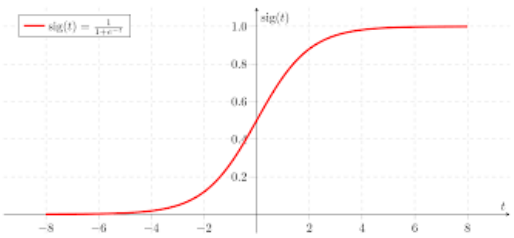


Fig. 10: Sigmoid function representation for the 6 red circles

So overall, GDA is more efficient (requires less data) if the Gaussian assumption is true, whereas Logistic Regression is more robust because it makes fewer assumptions about the data distribution.

1.11. ***Naive Bayes***

1.11.1. **Definition & assumptions**

If we stick with our example, we want to know how do we categorize an email when it arrives in our mailbox? First, we have to create a condition for the mail, we can for example create a vector with all the dictionary words (we could just take 10000 for our example), and create a zero-one vector where the values are 1 if the word is in the mail, 0 else. But we would need 2^{10000} possible values, which is too much.

So in Naive Bayes, we assume that the x's are the conditionnaly independant given y, so :

$$\begin{aligned} p(x_1, \dots, x_{10000}|y) &= p(x_1|y)p(x_2|x_1, y)p(x_3|x_1, x_2, y)...p(x_{10000}|...) \\ &= p(x_1|y)p(x_2|y)p(x_3|y)\dots p(x_{10000}|y) \\ &= \prod_{i=1}^{n} p(x_i|y) \end{aligned}$$

So as we can see with the independance assumption, we end up with simpler equation, so the probability of the word "aadvark" (2nd in dictionnary) does not depend on the word a (1st in dictionary), which is not true in a mathematical sense, but most of the time it's not so horrible so we can get away with it.

1.11.2. **Parameters**

The parameters of the Naive Bayes model characterize the relationship between the words in an email and its classification as spam or not spam:

- Word probability in spam emails: The chance of word j appearing given the email is spam.

$$\phi_{j|y=1} = p(x_j = 1|y = 1)$$

- Word probability in non-spam emails: The chance of word j appearing given the email is not spam.

$$\phi_{j|y=0} = p(x_j = 1|y = 0)$$

- The overall probability that your next email will be a spam email.

$$\phi_y = p(y = 1)$$

1.11.3. **Joint Likelihood**

We can now define the Joint Likelihood:

$$\mathcal{L}(\phi_y, \phi_{j|y}) = \prod_{i=1}^{m} p(x^{(i)}, y^{(i)}; \phi_y, \phi_{j|y})$$

Where

- This represents for all the spam mail ($y = 1$), what is the probability of seeing the word j (numerator activates if the word j is in the mail, so $x_j = 1$, and denominator activates if mail is spam).

$$\phi_{j|y=1} = \frac{\sum_{i=1}^{m} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 1\}}{\sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 1\}}$$

- And this is the mean of spam (frequency of spam), for example if there is 30 spam and $m = 100$ mails, then $\phi_y = 0, 3$:

$$\phi_y = \frac{\sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 1\}}{m}$$

1.11.4. **Laplace smoothing**

At prediction time, we have this formula expressing on numerator the probability of x^i knowing y (example with emails: probability the the word i is in the mail), and in denominator we have all the cases (mail is a spam or not).

$$P(y = 1|x) = \frac{P(x|y = 1)P(y = 1)}{P(x|y = 1)P(y = 1) + P(x|y = 0)P(y = 0)}$$

But we could encounter a problem if the word has not yet been received in the mail box, because it would mean that the probability of this word being a spam is 0, which is not a good approach, because first it would mean that this word i has a probability of 0 to be on a spam mail (not true), and it would lead us to a division by 0.

In order to avoid this problem, we are going to use the **Laplace smoothing** where we can add 1 for each count (positive or negative).

If we take for example a soccer team that has lost its last four matches, what is the probability to win its fifth match?

Let's take back the last equation, on the numerator we would have 1, and denominator 1 + 5, so the result would be 1/6, much more realistic than 0, even if they still lost their fifth match.

More generally:

$$x \in \{1, \dots, k\} P(x = j) = \frac{\sum_{j=1}^{m} \mathbb{1}\{x^{(i)} = j\} + 1}{m + k}$$

Let's now take back the Naive Bayes parameters and use Laplace smoothing to take away that probleme of division by 0:

$$\phi_{j|y=0} = \frac{\sum_{i=1}^{m} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 0\} + 1}{\sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 0\} + 2}$$

1.11.5. **Multivariate event model**

Before we used the “multivariate Bernoulli event model”, but if a word appears more than 1 time in the email then we are loosing information (so if x_{200} appears 4 times we are going to have this word only once in the x vector).

$$x = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 1 \\ \vdots \end{bmatrix} \begin{matrix} \text{a} \leftarrow 1 \\ \vdots \\ \text{buy} \leftarrow 800 \\ \vdots \\ \text{crypto} \leftarrow 1000 \\ \vdots \\ \text{now} \leftarrow 6200 \end{matrix}$$

Here if the mail is "Crypto! Buy crypto now!", we'll have the word "crypto" only once.

So let's use a new model called "multinomial event model", where the new vector x has the size of the number of words in the mail, n_i; so we go from a vector size 10000 to 4:

$$x = \begin{bmatrix} 1000 \\ 800 \\ 1000 \\ 6200 \end{bmatrix} \begin{matrix} \leftarrow \text{Crypto} \\ \leftarrow \text{buy} \\ \leftarrow \text{crypto} \\ \leftarrow \text{now} \end{matrix}$$

So if we want for exemple to rewrite the parameter that gives the chance of word j being k if $y = 0$: $\phi_{k|y=0} = P(x_j = k|y = 0)$, we could rewrite it as follows with Laplace smoothing:

$$\phi_{k|y=0} = \frac{\sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 0\} \sum_{j=1}^{n_i} \mathbb{1}\{x_j^{(i)} = k\} + \mathbf{1}}{\sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 0\} \cdot n_i + \mathbf{10000}}$$

1.12. ***Support Vector Machines (SVM)***

SVM is a method that allows us to create a non linear decision boundary between 2 classes. Before, with Linear Regression, we had x_1 and x_2 giving a straight hyperplane, now we will create a new feature vector:

$$\phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{bmatrix}$$

It will find an ellipse (better than straight line), but with higher dimension we get a more precise decision boundary (DB).

1.12.1. **Optimal Margin Classifier**

First we could ask ourselves, how the hyperplane that separates two classes is drown? We are choosing the straight where the average distance is greater. This distance is called the geometric margin.

Before, with LR, we had $h_\theta(x) = g(\theta^T x)$ where $\theta \in \mathbb{R}^{n+1}$, $x_0 = 1$, now we will have the following hypothesis function:

$$h_{w,b}(x) = g(w^T x + b)$$
$$w \in \mathbb{R}^n, \quad b \in \mathbb{R}$$

And we drop the $x_0 = 1$ convention.

To be more precise, the parameters are the sum of all the "weights",

$$\sum_{i=1}^{n} w_i x_i + b$$

$$\left.\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}\right\} \begin{matrix} b \\ w \end{matrix}$$

Now let's see the conditions. So the margin of an hyperplanee is defined by (w, b) given a training example $(x^{(i)}, y^{(i)})$. The margin is expressed as: :

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x^{(i)} + b)$$

- If $y^{(i)} = 1$, we want $w^T x^{(i)} + b \gg 0$
- If $y^{(i)} = -1$, we want $w^T x^{(i)} + b \ll 0$

So we want this to be very very large if $y = 1$ (far right in sigmoid function), and same for $y = -1$.

Then, let's define the worst training example that will help us build the hyper plan (we choose the smallest value, it means that it is next to the middle of the sigmoid):

$$\hat{\gamma} = \min_{i=1,\dots,m} \gamma^{(i)}$$

So finally, the geometric margin of a hyperplane parametrized by w and b is:

$$\gamma^{(i)} = \frac{\hat{\gamma}^{(i)}}{\|w\|}$$

To ensure that the margin represents a physical distance, we normalize the value with $\|w\|$. So we used the worst possible example to establish our geometric margin, so then we want for all the others point to have a margin greater than the geometric margin.

1.13. ***Kernels***

First, we define the weight vector w as a linear combination of the training examples:

$$\mathbf{w} = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)}$$

The original objective is to minimize the squared norm of w. We substitute the definition of w into the formula:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \rightarrow w^T w$$

Substituting the summation:

$$\min \frac{1}{2} \left(\sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} \right)^T \left(\sum_{j=1}^{m} \alpha_j y^{(j)} x^{(j)} \right)$$

This simplifies to a double summation focusing on the inner product of the input features:

$$\min \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} \mathbf{x^{(i)T} x^{(j)}}$$

Where we recognize the inner product notation:

$$\langle x, z \rangle = x^T z \text{ is the inner product, so: } \langle x^{(i)}, x^{(j)} \rangle$$

The original constraint ensures a functional margin of at least 1 for all training examples:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1$$

Substituting the definition of w into this constraint:

$$y^{(i)}\left(\left(\sum_j \alpha_j y^{(j)} x^{(j)}\right)^T x^{(i)} + b\right) \geq 1$$

Expanding the inner product inside the constraint:

$$y^{(i)}\left(\sum_j \alpha_j y^{(j)} \langle x^{(j)}, x^{(i)} \rangle + b\right) \geq 1$$

Thanks to convexe optimisation, we can simplify :

$$\alpha_i \geq 0$$

$$\sum_{i=1}^{m} y^{(i)} \alpha_i = 0$$

1.13.1. **Kernel Trick**

This derivation formalizes the Kernel Trick, allowing SVMs to operate in high-dimensional feature spaces efficiently.We define a mapping function ϕ that transforms our original input x from a low-dimensional space (e.g., 2D) into a very high-dimensional feature space (e.g., 100,000D or even infinite-D):

$$x \mapsto \phi(x)$$

Examples of mappings shown:

For 1D input: $x \mapsto \phi(x) = \begin{bmatrix} x \\ x^2 \\ x^3 \\ x^4 \end{bmatrix}$

For 2D input: $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \mapsto \phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_1 \\ x_1 x_2 \\ x_2 x_2 \\ \vdots \end{bmatrix}$

Instead of explicitly computing the high-dimensional vector $\phi(x)$, which is computationally expensive, we find a Kernel Function $K(x, z)$ that computes the inner product in that high-dimensional space directly:

$$K(x, z) = \phi(x)^T \phi(z)$$

This allows us to learn complex non-linear decision boundaries while keeping the computational cost equivalent to working in the original low-dimensional space.

1.13.2. Gaussian Kernel (RBF)

In practice, the most commonly used kernel is the **Gaussian Kernel**, also known as the **Radial Basis Function (RBF)** Kernel. It is mathematically defined as:

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

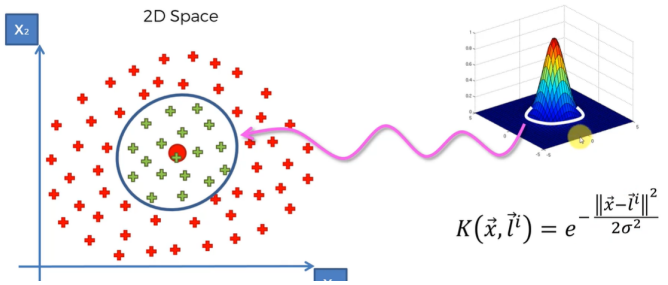


Fig. 11: Gaussian RBF Kernel in non linear SVM

This kernel is highly effective due to several key properties:

- **Similarity Measure:** The function $K(x, z)$ acts as a measure of proximity between two points x and z. The resulting value scales between 0 and 1:
  - When $\|x - z\| \approx 0$ (points are very close), then $K(x, z) \approx 1$.
  - When $\|x - z\|$ is large (points are far apart), then $K(x, z) \approx 0$.
- **Infinite Dimensionality:** Mathematically, the Gaussian kernel corresponds to an implicit feature mapping $\phi(x)$ into an *infinite-dimensional* space. This allows the SVM to learn decision boundaries of arbitrary complexity that would be impossible to define in the original space.
- **The σ Parameter (Bandwidth):** This parameter controls the "reach" or influence of a single training example:
  - **Small σ:** The kernel falls off quickly. The model only considers very close neighbors, leading to a more complex and potentially *overfitted* boundary (high variance).
  - **Large σ:** The kernel falls off slowly. The influence of each point spreads further, resulting in a smoother, more *generalized* boundary (high bias).

1.14. Data Splits, Models & Cross-Validation

1.14.1. Regularization

A common challenge in machine learning, particularly with Logistic Regression and high-dimensional models, is the risk of **overfitting**. This occurs when the model becomes too complex, capturing the noise and random fluctuations of the training data rather than the underlying pattern. To address this, we introduce **Regularization**.

The core idea is to discourage the model from learning excessively large parameters θ. We modify the cost function $J(\theta)$ by adding a penalty term, typically the L_2 norm:

$$J(\theta)_{reg} = J(\theta) + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

The regularization parameter λ (lambda) controls the trade-off between two competing goals:

– **Fitting the training data well:** Keeping $J(\theta)$ low.
– **Keeping the model simple:** Keeping the parameters θ small to prevent wild oscillations.

As shown in the figure below, adjusting λ allows us to navigate between *High Variance* (overfitting) and *High Bias* (underfitting).

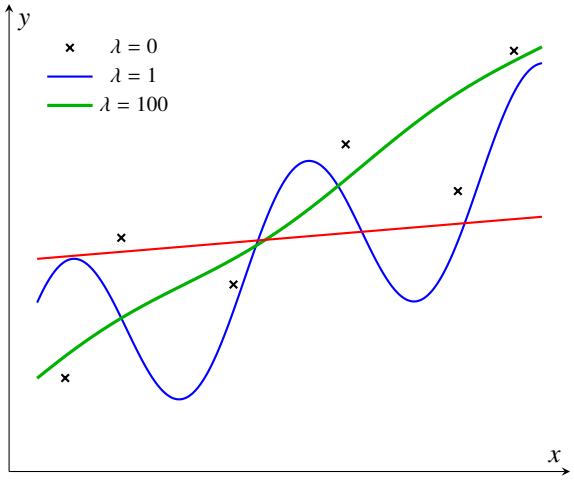


Fig. 12: Impact of the regularization parameter λ.

a

References

[1] A. Ng, *Machine Learning Specialization*, Stanford Online / Coursera, 2024. [online]. Available at: https://www.coursera.org/specializations/machine-learning-introduction

[2] S. Pawar, *Physics-Informed Neural Networks for Fluid Dynamics*, Georgia Institute of Technology, 2023.

[3] NumPy Developers, *NumPy Documentation: Matrix Operations and Vectorization*, 2025. [Online].